Tottie

Self-Curating Agent Memory over MCP

Frans Nekongo

Meta Global AI Developer Hackathon
Indie Developer Track

Nov 2026

A year of conversations. It still doesn't know your name.

The real problem

Agents do not have a reasoning problem. They have a **state persistence** problem.

- Every agent wakes up stateless: no history, no context, no past.
- A bigger context window is not memory — it is an expensive scratchpad.
- Longer context $\Rightarrow$ linearly higher cost, super-linear latency, *lower* accuracy.

"All that intelligence, and it fails the baseline of a mind: continuity."

Everyone searches at read time. We think at write time.

Standard RAG

Intelligence at **read** time: the agent searches when it needs something. Memory is a pile of notes the agent must juggle.

Tottie

Intelligence at **write** time: memory curates itself as you live. The agent does not manage memory — it just remembers.

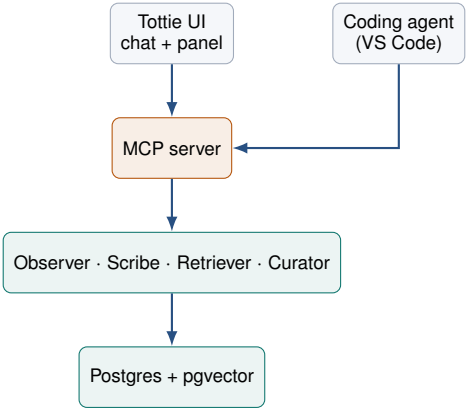
This is not a “basic RAG application.” The value is the **ingestion-time decision** and the deliverable is an **MCP server**.

What Tottie is

A self-curating memory architecture that runs the memory lifecycle **in parallel** with the agent, and ships as one **MCP server** any agent can mount.

- Positioned against 2026's **learned consolidation** field (mem0, Zep-Graphiti, A-Mem).
- Differentiator 1: write-time salience + **reconciliation** is the product, not a bolt-on.
- Differentiator 2: **transport-agnostic** — the same brain in a chat, an IDE, anything.

Four minds. One job each.



- **Observer** — is this worth remembering?
- **Scribe** — structure, tag, embed, **reconcile**.
- **Retriever** — runs every turn; find what is contextually relevant.
- **Curator** — write the compact **Context Card** the agent receives.

The memory vault has tiers

Tier	What it holds
Identity	Persona + permanent constraints. Pinned raw, zero search.
Episodic	Chronological decisions, state changes, history.
Knowledge	Ingested docs, files, references.
Procedural (“Lessons”)	Mistake $\rightarrow$ solution pairs. Autosave.

Why tiers beat a flat vector store

Identity needs no search at all; episodic needs time; knowledge needs semantics; lessons need an error signature. One store, four retrieval policies.

The part everyone leaves out: what reaches the model?

Dual-path recall: a card that is handed over automatically, plus a probe the agent can choose to run.

The Context Card (injected every turn)

```
[MEMORY CARD - 3 hits]
▷ IDENTITY (pinned) user=Frans - girlfriend birthday = Nov 7
▷ EPISODIC 2026-10-29 decided:  pgvector, not Qdrant #tottie #decision
▷ KNOWLEDGE (telecom-arch.pdf p.7) Server C behind A/B #telecom #infra
→ probe deeper:  memory.probe("#telecom")
```

- **Ambient** — the card is always there; the agent "just knows."
- **Deliberate** — the card carries pointers; `memory.probe(tag)` opens the room.

It changes its mind, correctly

Contradiction at write time, not read time

"I'm vegetarian now" $\rightarrow$ the Scribe **supersedes** the old fact: old row `valid_to` $=$ *now*, new row `valid_from` $=$ *now*.

- Retrieval filters `valid_to IS NULL`, so a stale fact can never surface.
- Plain vector recall has *no update semantics* — both facts return and the model guesses.
- This is exactly LongMemEval's *knowledge update* and MemoryAgentBench's *conflict resolution*.

One brain. Three bodies.

Tottie

Chat + reacting sprite.
Remembers your *life*.

Coding agent

Same MCP server in VS Code.
Knows your *stack*; autosaves fixes.

Memory Panel

The live X-ray. Watch it think.

The money shot: add a fact in the chat, then ask your coding agent — it already knows.

The Memory Panel: the invisible, made visible

Real time, four blocks:

1. **Learning now** — the turn + Observer verdict (KEEP/DISCARD, salience, reason).
2. **Memories formed** — `NEW` / `UPDATE` / `SUPERSEDE` / `SKIP`, with the text.
3. **Inserted where** — tier chip + tag/room chips + a growing room map.
4. **Context injected** — the exact card the model received this turn.

Why this is the hero

Judges should not have to take “it remembers” on faith. They watch the decision happen.

We do not ask you to trust it. We measure it.

- A $\sim$**40-item** suite across LongMemEval's categories (knowledge update, temporal, preference) plus contradiction resolution and abstention.
- Every item runs **memory ON vs OFF** — the ablation that proves memory caused the answer.
- Metrics: answer accuracy, memory-hit@k, contradiction-correctness, tokens/query, p50/p95 latency.

The headline

91% lower p95 latency and **90% fewer tokens** than full-context, for a small accuracy trade — the write-time curation argument, made falsifiable.

What was built during the Hackathon (Oct 27 – Nov 4)

Component	
`core/`	Memory engine: Observer, Scribe, Retriever, Curator; pgvector store; reconcile
`mcp/`	MCP server: 6 tools, 2 resources, the context-card prompt
`ui/`	Tottie chat, sprite state machine, live Memory Panel, ablation toggle
`eval/`	ON/OFF ablation harness
`docs/`	Full architecture + flows + this deck (LaTeX $\rightarrow$ PDF)

Every component is committed on a dated branch; the ledger is in `docs/business/hackathon_compliance`.

So what did I actually build it for?

"Everything I was mad at my girlfriend for —
and the one thing that matters: her birthday."

- **Autosave** — an agent makes a mistake once, and never again.
- **A personal AI** that actually knows you, not just your last 20 messages.
- **The company brain** — remember that Server C sits behind A and B before designing it.

Tottie

It just remembers.

github.com/frans-nekongo/Tottie